INTEGRATION, the VLSI journal 90 (2023) 261–270

Contents lists available at ScienceDirect

Integration, the VLSI Journal

journal homepage: www.elsevier.com/locate/vlsi

Machine learning application for cell delay accuracy improvement at post-placement stage: A case study for combinational cells

Yassine Attaoui [a,*], Mohamed Chentouf [a], Zine El Abidine Alaoui Ismaili [b], Aimad El Mourabit [c]

[a] Siemens EDA/CSD Calypto - Synthesis Solutions, Rabat, Morocco
[b] Information, Communication and Embedded Systems (ICES) Team, Mohammed V University, Rabat, Morocco
[c] System & Data Engineering Team, ENSA of Tangier, Abdelmalek Essaâdi University, Tangier, Morocco

ARTICLE INFO

Keywords:
Delay estimation
Machine learning
Timing accuracy
Regression
Cell delay

ABSTRACT

In the early VLSI design stages, as most interest is given to timing closure, various techniques are deployed to satisfy the timing constraints and close design timing. Nevertheless, timing still needs to be accurate at the early stages due to the lack of physical design information. Therefore, the challenge for most EDA Logic Synthesis engines is to improve the overall timing accuracy at higher levels of abstraction. In this paper, we shall reduce the error gap in *Cell Delay* estimation for a placed gate-level netlist. We will generate the post-placed netlist from two different *Siemens EDA* tools. The purpose is to improve the accuracy of the cell delays of *Oasys-RTL* Synthesis tool, based on accurate cell delays from *Aprisa PnR*. We have trained and tested various Machine Learning models on several 16 nm designs. We achieved the highest accuracy using the *Random Forest* model, where the average accuracy is 91.25%. Furthermore, as the accuracy of the *Cell delay* is improved, the average Root-Mean-Square-Error (RMSE) is reduced from 16.958 to 4.469.

1. Introduction

Nowadays, VLSI systems are increasingly complex, and the timing closure evolves as a challenge after each step of the development flow. The circuit's timing closure at a high level of abstraction has always been the focus of all EDA companies after ensuring the circuit's functionality. To make sure the design timing behavior is within the timing budget and meets all timing constraints at a defined operating frequency, it is mandatory that EDA tools first estimate accurate cell and interconnect delays. This study will focus on the *Logic Synthesis* stage, where most EDA tools misestimate cell and net delays at Post-Synthesized gate-level netlist. Usually, path delays are either overestimated or underestimated compared to the post-placed netlist generated from a powerful P&R tool. This inaccuracy in the cell and interconnect delays reduces by advancing in the physical design flow, where the layout information is no longer estimated.

In this paper, we will attempt to improve *Oasys-RTL Synthesis tool (Siemens EDA)* [1] cell delay accuracy with the help of Machine Learning algorithms. We will perform a prediction of each cell delay and compare it with the golden equivalent cell delay using *Regression* ML models. For this purpose, we considered the same cell delays extracted from *Aprisa PnR (Siemens EDA)* [2] as golden data. We intend to improve the timing accuracy at a preliminary stage at the post-synthesized gate-level netlist. This task will greatly impact subsequent design flow steps and the time-to-market factor, as it will reduce the iterative process on ASIC design, especially during placement [3].

The *Oasys-RTL* performs high-level timing optimization and iterative logic optimizations to generate an optimized placed netlist that meets the timing and area budgets. A third-party PnR tool can use the Oasys-generated timing-aware placed netlist for further physical design steps. *Oasys-RTL* uses NLDM extraction to estimate standard cell delay, along with the wire–load model for net delay estimation [1,2]. The tool occasionally miscalculates cell delays, and the average error gap between the Aprisa final and Oasys post-placed netlists remains slightly higher. More accuracy is achieved when the SPEF file with accurate RC parasitics values is provided. However, generating a netlist using an already generated SPEF file can take several days or weeks due to the SPEF generation and the design runtime, which depends on the size and architecture of the design.

Recently, Machine learning (ML) applications are increasingly involved in VLSI CAD and ASIC design, and various EDA-based companies are adopting ML for different development purposes. Nonetheless, ML still needs to be fully exploited in the VLSI environment. In addition, the amount of data within the development flow continuously increases as we go down the flow, where additional layout and physical information evolve. Therefore, ML is a promising subject as we shall use it to

* Corresponding author.
E-mail addresses: yassine.attaoui@siemens.com, yassineattaoui@gmail.com (Y. Attaoui).

https://doi.org/10.1016/j.vlsi.2023.02.011
Received 11 August 2022; Received in revised form 14 February 2023; Accepted 23 February 2023
Available online 25 February 2023
0167-9260/© 2023 Elsevier B.V. All rights reserved.

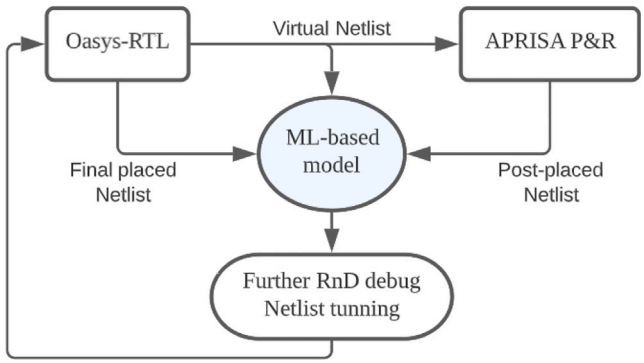

Fig. 1. ML-based Oasys-RTL metrics improvement.

explore the circuit dataset and reveal some features, correlations, and dependencies, hence improving the tool's QoR.

Our primary concern is applying various ML models to improve cells' delay for Oasys-RTL, considering the extracted cells' delay from Aprisa P&R. Our approach consists of running 5 Regression algorithms on a set of designs. We will train the ML models to reduce the gap error on cell delays and test and evaluate their accuracy and error rate afterward. We have taken much care to keep similar post-placed netlists for both tools, while each tool performs placement and optimizations differently. First, the *Oasys Virtual-Netlist* is generated and then fed to *Aprisa* for further optimizations. In the meantime, Oasys will perform its optimizations to generate *Oasys Final Netlist*. Finally, cell delays will be extracted from the final Oasys netlist and Aprisa post-placed netlist as in Fig. 1.

We have split the remainder of this paper into four major sections. Section 1 provides state of the art on ML applications on VLSI design and related works. Section 2 presents the feature extraction and provides an overview of the dataset. Next, Section 3 gives some ML background, how we proceed with features and data processing, model training, testing, and evaluating. Finally, in Section 5, we present the experimental results and conclude afterward.

2. Related work

Recently, Machine Learning applications have been increasingly growing in VLSI. Various EDA and IC design companies are exploring ML approaches that reduce the time-to-market and manufacturing runtime while assuring IC functionality. As a result, numerous applications are used at different design stages and using diverse approaches. This section studies various state-of-the-art ML applications related to VLSI Power, Performance, Area (PPA), and routing improvement.

[4] presents a practical ML application for area prediction from a high-level specification of RTL components. The model achieved a high accuracy regardless of lacking netlist information at the specification level. From the HLS stage, [5] implemented a model to predict routing congestion. The routing bottlenecks were determined at early stages and resolved from the RTL level. In [6], the author tries to review power-grid-related issues such as IR drop and Electromigration (EM) using ML approaches. It implements a Machine Learning model that predicts the EM aging of the power grid network, as well as predicts a reliable power grid network by predicting the widths of metal lines. In terms of power analysis, [7] suggested an ML model that achieves high power estimation accuracy for RTL blocks across different benchmark designs. Compared to commercial gate-level power analysis tools, the author's model reached satisfying accuracy and performance. In [8], the author suggested an ML model that improves the switching power accuracy. Various runs are conducted using (SPEF) file back-annotated RC parasitics. The trained model predicts the cell's switching power without needing a SPEF file *spefless flow*, which has achieved good accuracy and reduced error on switching power.

In [9], the author proposes a predictive machine learning model for PBA (Path-Based timing Analysis) from GBA (Graph-Based timing Analysis). The model reduces the gap of PBA-GBA divergence consisting of arrival time and slack accuracy. Furthermore, since cell delay estimated using GBA-mode is more pessimistic than PBA-mode, the model arrives to reduce the PBA-GBA timing pessimism within a reduced runtime relative to ordinary PBA-mode. The model has achieved a well timing–runtime tradeoff with reduced predictive arrival time divergence and lower runtime.

The author in [10] leverages the timing uncertainty at the pre-routing stage by implementing the Random Forest algorithm. The model has largely reduced the pessimism on net delays with improved pre-routing net delay predictions. Our study aims to bridge the gap between the pre and post-placement stages by utilizing machine learning techniques. Our approach also extends the accuracy of the design into the early stages of the design process, precisely the Synthesis stage. Unlike [10], which utilizes small designs with a maximum of 1400 flip-flops, our approach addresses the real challenges and difficulties encountered in actual System-on-Chip (SoC) implementations.

A similar approach is conducted in [11] to improve timing accuracy at the pre-route stage. The author proposed a GNN-based (Graph Neural Network) machine learning framework to correct the arrival time and slack at endpoints. The timing engine GNN-based model has reached accurate pre-routing net delay and pin arrival time within a short inference time compared to the complete routing and STA flow. The authors worked on small and medium-sized designs and predicted routing impact on timing to guide the placement at the physical implementation. However, the feature set used for prediction was kept minimal.

[12] proposes an ML-based routing-free crosstalk prediction framework. The model classifies nets that may have large coupling capacitance or crosstalk noise. The authors predicted routing impact on timing to guide the placement at the physical implementation. In addition to improving timing at the routing stage, the authors also utilized small-sized designs, and few features were extracted.

In [13], the author tries to minimize the divergence in path slack between an internal and a signoff STA tool. The model developed does not include cell delay miscorrelations. However, in [14], the author developed a learning-based approach to correlate the timing divergence for path slack, setup time, stage, cell, and wire delays. The experiment is conducted on 45 nm and 28 nm. The model corrects slack divergence that reaches a reduction of up to 7 times.

This work introduces a machine learning approach to pre-place timing improvement of a synthesis tool. The delay predictions reach an accuracy near the post-place timing delay of a physical-aware P&R tool. We largely avoid pessimism by using ML to extract knowledge from a physical-aware timing engine. Our main idea is to decompose the path delay into a cell and net delay and focus on training the model to predict solely combinational cell delays. The predictions are timing-arc based, and the model predicts based on the cell unateness. We trained the model taking into account the *timing_sense* attribute. Our method significantly advances over previous works as it closes the gap between pre- and post-placement, taking accuracy further to the earlier Synthesis stage. As designs are getting more complex with billions of gates in a single SoC, the development and fabrication costs are increasing, and the PPA target is pushed to the limits. Improving the timing accuracy at upper development stages will help meet the PPA target without respine, reducing the development cost and time to market. Additionally, this work utilizes larger and real industrial technology node designs. Finally, while implementing various ML models, we extracted a large number of cell attributes around 101 features. As a result, the proposed model reaches considerable delay prediction accuracy on advanced technology nodes. The experimental results show that our approach can obtain a 91.25% delay accuracy and 4.469 Root-Mean-Squared-Error.

The contributions of the current work are as follows.

- To the best of our knowledge, this is the first work to present a machine learning framework for predicting post-place cell delays using advanced 16 nm technology designs.
- The model's predictions are timing-arc based for post-placed combinational cells. In addition, the Models are trained to predict cell rise delay and fall delay separately.
- The ML model has potentially reduced the delay uncertainty gap with an average accuracy of 91.25%. Furthermore, the model predicts within a short inference time.
- The proposed model exhibits good generalization performance, and its accuracy can be further improved by incorporating additional training data and tuning hyperparameters.

The ML application on VLSI design is eventually a raw subject, and additional research efforts can be conducted in terms of QoR enhancement [15]. In the next section, we present our intent in applying ML to enhance the cell delay accuracy for Oasys-RTL gate-level netlist. The tool's inaccuracy at the logic synthesis stage is mainly due to the lack of physical information. This leads to challenging timing closure at high abstraction levels, and timing-critical paths may reside violated due to miscalculations. Moreover, the tool's inaccuracy may lead to a false violation or a false non-violated path. In this trial, we attempt to improve the accuracy of cell delays by feeding-back physical information to our ML models as in Fig. 1.

3. Designs characterization

This section presents the flow used to generate the Oasys-RTL final netlist and the Aprisa placed netlist. We also briefly describe the implemented designs and an overview of the used ML algorithms.

We have performed two separate runs using five multi-VT industrial designs of different sizes as presented in Table 1. The first run mainly uses *Oasys-RTL* synthesis tool to generate a post-place netlist. The second run uses *Aprisa* PnR to generate a similarly optimized netlist. We summarize designs' information in Table 1. The total number of standard cells reflects the size of each design. The extracted data is split into training and testing datasets for TC1 to TC4, while we used TC5 entirely for testing purposes.

As the clock network at this stage is ideal clock, the slew at the Flip-Flop inputs is considered null compared to the combinational cell's inputs. Mixing sequential and combinational timing arcs will not help the ML models train and learn. We have split the problem into two categories *Sequential* and *Combinational*, as they have different properties. We focused on *Combinational cells delay*, while the sequential cells delay will be subject to another study. In addition to sequential cells, net delays are discarded and will be studied in a separate paper. We instructed *Oasys-RTL* and *Aprisa* to run using *0-RC* and *0-LOAD* parameters consecutively, which will consider nets as ideal interconnects with zero delays.

The flow commands presented in Algorithm 1 and Algorithm 2 set design initial setup files, logical and physical libraries, process technology libraries, design constraints, and a predefined Floorplan. Afterward, the logic is mapped to technology cells and timing-aware area optimized. The timing paths are then defined and optimized. This step is iterable until optimizing all possible timing violated paths (few paths may remain violated). Then, we instructed *Oasys-RTL* to generate a *Virtual Netlist*. Finally, the tool performs a timing-aware placement to generate a final netlist for further backend process [1,2]. Next, we collected the designs' data in CSV files. The CSV files contain the cell's and net's and library cell's attributes, corresponding cell delays, and QoR metrics for all possible design timing arcs (Algorithm 1).

On the *Aprisa* side, we have used the already generated *Oasys-RTL Virtual Netlist* as a starting netlist. Afterward, we instruct the tool to place the design with low effort to avoid changing the netlist. Finally, from the final post-placed netlist, we extracted QoR metrics and cell and net properties (Algorithm 2). The objective here is to improve the timing estimation at the synthesis phase. We need minimal changes in the Aprisa placed netlist to measure the timing inaccuracy induced by the lack of physical parameters, such as wire length, net resistivity, and capacitance. We will tune the synthesis STA engine to correlate with the PnR STA estimations. Thus, Oasys sees and optimizes real timing violations, improving flow convergence and runtime at the physical design step. This objective requires the PnR to place and globally route without optimizing the design, then estimate the timing by the PnR static timing analysis engine and measure the differences compared to synthesis reported values. Hence, we called the core placement engine of Aprisa with low timing effort. We aim to integrate the trained model in *Oasys-RTL* environment. Therefore we avoid *Aprisa* features and backend attributes dependencies during model training.

Table 1
Designs features.

Design	Node (nm)	#Clocks	Freq. (Ghz)	#Std cells	#Macro	Area (squm)	#FF	#Comb.
TC1	16	2	2	2274	0	1100	413	**1611**
TC2	16	2	1.4	30366	0	19134	9424	**20497**
TC3	16	2	1.6	38826	0	44501	12146	**24567**
TC4	16	2	1.4	109160	0	51477	19388	**78153**
TC5	16	2	1.3	73759	0	47578	12455	**51804**

Algorithm 1 Oasys-RTL Frontend flow.

```
Import design data
Set 0-RC parameter
Set design timing constraints
Import floorplan
Synthesize design, map to technology cells
Perform Virtual optimizations
Write Virtual Netlist
Optimize timing paths groups
Perform Placement
Perform post-placement opt.
Write Final Netlist
Report cells attributes
Report nets attributes
Report libraryCells attributes
Report Cells delays and QoR metrics
```

Algorithm 2 APRISA Backend flow.

```
Import design data
Set config timing 0-LOAD mode
Set design timing constraints
Import floorplan
Import Oasys-RTL Virtual Netlist
Perform global and detailed placement
Perform global route
Write post-placed Netlist
Report cells attributes
Report nets attributes
Report libCells attributes
Report cells delays and QoR metrics
```

4. Machine learning algorithms and dataset

Machine Learning allows the user to recognize pattern correlations and features' trends within complex data structures. This trial aims to apply ML to reveal the patterns between the extracted cell delay and other cell attributes. As Backend data continuously increases, manually revealing the features correlations by human observation becomes almost unfeasible. We assume the predicted data is a continuous numerical float for cell delay prediction. Therefore we require a *SUPERVISED*

Table 2
Outliers percentage on Dataframe.

Design	Total	Outliers	Outliers (%)	Remaining
TC1	4621	60	1.29%	4561
TC2	78462	1	0.00%	78461
TC3	94913	4	0.00%	94909
TC4	188753	366	0.19%	188387

Regression model. Various state-of-art *Regression* models were studied. The chosen algorithms are *Linear Regression, Decision Tree, Random Forest, Support Vector Regression*, and *K-Nearest Neighbors*. The role of data in ML is crucial, and the quality of data is the key to improving the model's accuracy. Consequently, we spent considerable time on the dataset processing. This section provides an overview of the used models, data processing, and model evaluation.

4.1. Data processing

In this chapter, we present different data pre-processing steps. We collected data by generating a collection of CSV files as in the flow in Algorithm 1 and Algorithm 2. Then, the CSV files corresponding to cell and net properties, cell-net connections, library cell attributes, cell delays, and QoR metrics were merged into one master CSV using inner juncture on the cell *full name*, *from pin* and *to pin* columns. Where the *full name*, *from pin* and *to pin* are specified as data-frame indexes (ID) of the master CSV and identify each timing arc. During data preparation, we dropped several columns from the master CSV due to their weak informative contribution to the prediction. In the whole pre-processing step, the Scikit-learn Pandas library [16,17] is used to manage the master CSV file as a Dataframe that represents all valid design timing arcs. Table 3 summarizes the Dataframe's information used for the training and testing datasets. Every row of the Dataframe represents a timing arc, and every column is an attribute of the corresponding cell.

As Aprisa golden data divides timing arcs into positive and negative unate, referred to as *cell delay rise* and *cell delay fall* attributes. Consequently, we trained the ML models first to predict *delay rise* attribute, which refers to positive unate timing arcs. Then, we carried a similar approach to predict *delay fall*. This paper contains the result accomplished on cell delay rise prediction, while delay fall prediction follows a similar approach.

As part of data processing, outlier removal is a preliminary phase in data preparation. Outliers are anomalies in the dataset caused by the tool's miscalculations due to the lack of information or errors while manipulating data. We started detecting outliers by plotting a scattered plot and identifying cells with massive output capacitance and slew differences. The cut-off using Inter Quartile Ranger (IQR) approach at 6-sigma has defined the upper and lower bounds from the mean value. We discarded only cells located beyond and below 6-sigma to drop the fewer possible outliers and keep data integrity while achieving higher accuracy. In Table 2, we summarize the outliers percentage per design. In Fig. 2, we display a scattered plotting after outlier removal, the X-axis is the *Oasys cell delay*, and Y-axis is the *Aprisa cell delay*. We also highlight the output capacitance as a heatmap. Finally, these outlier timing arcs were dumped from the TC1, TC2, TC3, and TC4 dataframes. However, TC5 is considered new raw data, and no outlier was removed. We remarked that these outliers are High Fanout cells and will be treated in a separate paper using clustering algorithms.

The *feature selection* is also a component of data preprocessing. Its purpose is to improve the model performance by reducing the number of input variables. The feature selection evaluates the correlation of each input variable with the outputs variable. The most contributing input features are given a score depending on their correlation level. Features with higher scores are the most contributing to the output variable prediction. We performed a *RFE* (Recursive Feature Elimination) as a feature selection technique in this study. However, for *Random Forest* and *Decision Tree*, the feature elimination is done automatically [18–20].

The next step is the *feature encoding*, otherwise converting categorical columns of a class object into numerical or boolean columns. The gathered datasets contain diverse categorical input variables. For instance, *Arc Sense* and *Arc Type* columns take a few states solely and should be treated as *enumerate* type. Therefore we converted them into boolean type by creating new columns. The number of added dummy columns depends on the number of states.

As a result, the dataframes after pre-processing have kept integers, boolean and floating types, with no outlier and all categorical types converted into boolean columns.

4.2. Model training

The shortlisted models were adapted and fitted to our datasets using Python Scikitlearn library [18]. In this chapter, we give a slight background of each model.

Multi-Linear Regression: The MLR is also known as multiple regression, which is an extension of single linear regression. This algorithm predicts continuous variables from multiple linear combinations of input variables. The algorithm defines a set of coefficients (weights) for each input variable. Then, it keeps correcting the coefficients based on the resulting accuracy of the predicted output variable [18,21].

Decision Tree: The DT is a non-linear algorithm that uses a tree model of decisions. DT is also used as regression to predict continuous data and is remarkably powerful that can fit complex datasets. The tree of decisions contains nodes and leaves, the nodes are tests on the attributes, and the leaves are the results of the tests [16,18,22].

Random Forest: Also known as the Random Decision Forests, the RF is one of the powerful algorithms for data prediction. The RF is constructed from multiple decision trees, and the resulting outputs of each decision tree will help the RF predict the output variable value. Using multiple trees makes RF a well-fitting algorithm for most kinds of datasets [18,22].

Support Vector Regression: The regression part of the Support Vector Machine (SVM) can also be used to predict continuous data. Its mission is to approximate a large number of data to a linear function. However, the difference from linear regression is that the SVR sets a threshold error allowance around the function line, which authorizes and considers points within the threshold margins [18].

K-Nearest Neighbors: The KNN is a non-parametric algorithm also used for regression. The KNN's purpose is to find similarities between the input and output variables. Then it selects the K closest inputs to build the non-linear function. The KNN weakness is the runtime, and the algorithm becomes slower while dealing with a large dataset [18].

4.3. Model evaluation

Once the model is fitted to the dataset and trained to predict new data, we evaluate the quality of predictions since the purpose is not only to build a model but to create a high-quality model with high predictive accuracy. We have chosen the *Root-Mean-Squared-Error* and *Mean-Absolute-Error* as metrics to measure the error rate of each model, and we used *R2 score* to evaluate how good the predictions are. We compared each model's performance with these metrics and drew a conclusion in the next section [18].

Root Mean Squared Error: the RMSE refers to the squared root of the sum of the squared distances between predicted and actual values (Eq. (1)) [18].

$$RMSE = \sqrt{\frac{\sum_{i=1}^{n}(predicted_i - actual_i)^2}{n}} \tag{1}$$

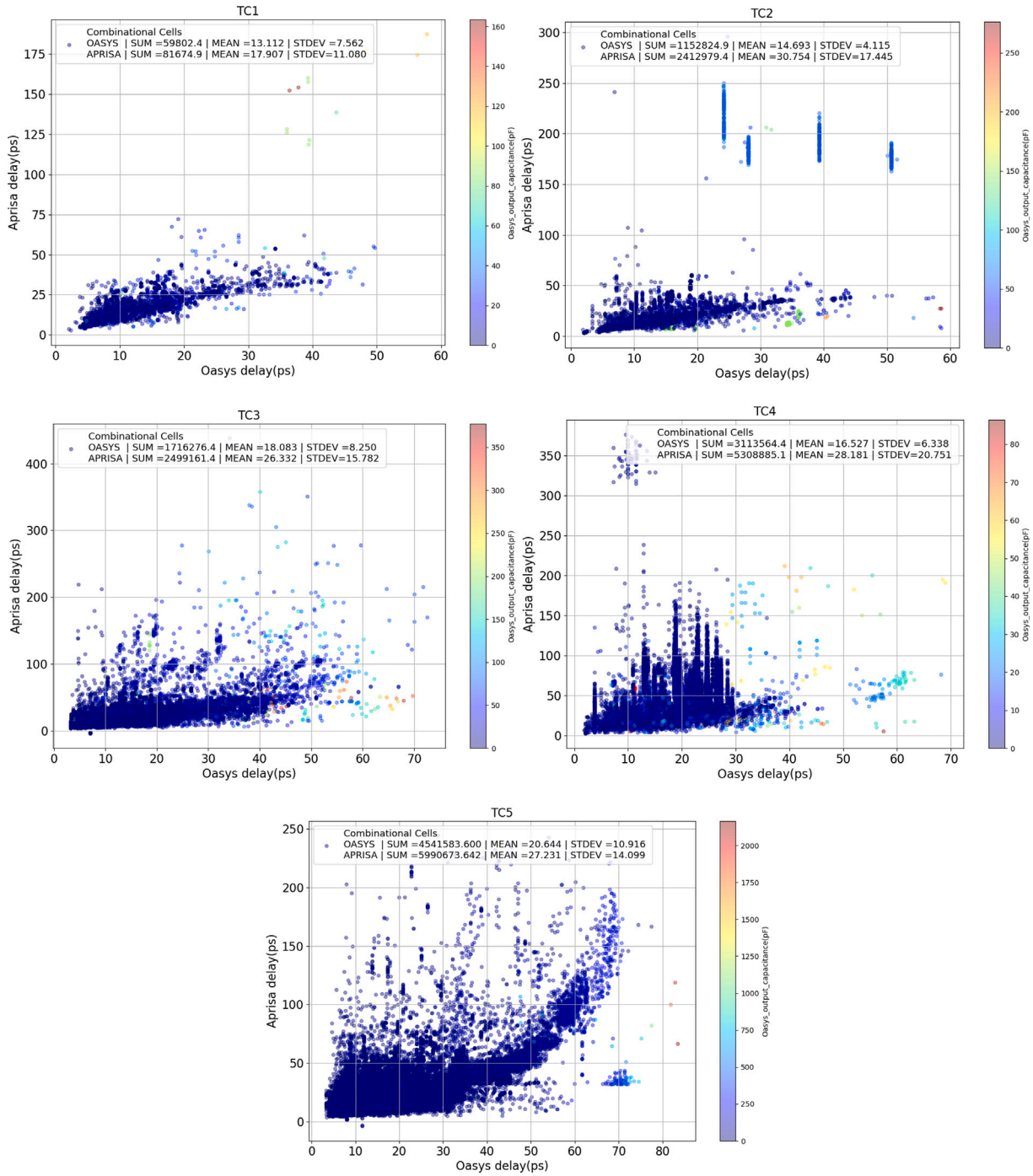

Fig. 2. Scattered plots for initial Oasys cell delay versus Aprisa cell delay.

Table 3
Design Dataframes.

Design	Rows°			Columns°				Model purpose	Oasys cell delay			Aprisa cell delay	
	Training	Testing	Total	Int64	Float64	Bool.	Total		MEAN	STDEV	RMSE	MEAN	STDEV
TC1	3420	1141	4561	19	18	64	101	Training/Testing	13.112	7.562	9.083	17.907	11.080
TC2	58845	19616	78461					Training/Testing	14.693	4.115	22.525	30.754	17.445
TC3	71181	23728	94909					Training/Testing	18.083	8.250	14.785	26.332	15.782
TC4	141290	47097	188387					Training/Testing	16.527	6.338	21.442	28.181	20.751
Average	–	–	–	–	–	–	–	–	15.603	6.566	**16.958**	25.793	16.264

Table 4
Bunch Dataframe.

Design	Rows°			Columns°				Model purpose	Oasys cell delay			Aprisa cell delay	
	Training	Testing	Total	Int64	Float64	Bool.	Total		MEAN	STDEV	RMSE	MEAN	STDEV
Bunch	274738	91580	366318	19	18	64	101	Training/Validation	16.495	6.642	**20.069**	28.125	18.877
TC5	0	142489	142489					Testing	20.644	10.916	12.463	27.231	14.099

Table 5
RMSE, MAE and R2 for models trained per design.

Design	MLR			DT			RF			KNN			SVR		
	R2(%)	RMSE	MAE	R2(%)	RMSE	MAE	R2(%)	RMSE	MAE	R2(%)	RMSE	MAE	R2(%)	RMSE	MAE
TC1	73.39	5.481	3.709	79.01	4.868	2.490	87.47	3.761	1.483	85.20	4.088	1.817	77.31	5.062	2.855
TC2	64.00	10.097	5.541	97.58	2.614	1.145	98.06	2.343	1.000	98.11	2.378	1.126	87.41	5.989	1.766
TC3	48.18	11.310	6.445	87.99	5.443	2.643	92.29	4.362	1.646	90.06	4.951	1.809	82.67	6.541	3.259
TC4	59.26	13.234	7.621	85.39	7.923	3.840	87.21	7.413	3.474	83.78	8.349	3.824	74.29	10.513	4.066
Average	61.20	10.030	5.829	87.49	5.212	2.529	**91.25**	**4.469**	**1.900**	89.28	4.941	2.144	80.42	7.026	2.986

Table 6
Models RMSE, MAE and R2 for model trained using Bunch dataset.

Design	MLR			DT			RF			KNN			SVR		
	R2(%)	RMSE	MAE	R2(%)	RMSE	MAE	R2(%)	RMSE	MAE	R2(%)	RMSE	MAE	R2(%)	RMSE	MAE
Bunch (validation)	47.37	13.838	7.994	85.41	7.285	3.391	**89.12**	**6.290**	**2.731**	86.67	6.963	2.884	85.86	6.285	3.505
TC5	43.95	10.556	5.300	56.09	9.343	4.083	**75.72**	**6.947**	**3.099**	67.98	7.979	3.877	59.32	8.993	4.453

Mean Absolute Error: The MAE is the sum of all absolute error margins of the predicted value and the actual values (Eq. (2)) [18].

$$MAE = \frac{\sum_{i=1}^{n} |predicted_i - actual_i|}{n} \tag{2}$$

R2 score: The coefficient of determination R2 describes the sum of squared residuals per sum of squares. The sum of squares is the distance from the actual and their average. While the residuals are the distance between the predicted and actual values. The R2 score is a percentage that indicates the predicted responses are good when R2 is closer to 1 (Eq. (3)) [18].

$$R2 = 1 - \frac{\sum_{i=1}^{n}(actual_i - predicted_i)^2}{\sum_{i=1}^{n}(actual_i - mean)^2} \tag{3}$$

5. Experimental results

In this section, we present the results of each model after training and testing. We first compared Oasys' initial delays versus the Aprisa golden delays (Fig. 2). Second, we plotted a graphical representation of the predicted cell delays versus Aprisa golden delays (Fig. 3). Finally, we analyzed the plots and identified the adequate model with the highest accuracy.

5.1. Data analysis

First, we visualized the initial Oasys versus Aprisa cell delays in scattered plotting, as pictured in Fig. 2. The X-axis is the initial state of Oasys cell delay, and the Y-axis is the corresponding Aprisa cell delay. For a more reasonable interpretation, we comprised the cell output capacitance referred to as the heatmap color for each cell. The higher the capacitance is, the more the color is red. The scattered dots visualize a slight inaccuracy of Oasys-RTL in calculating cell delay. Next, the MEAN and STDEV of the Oasys cell delays are computed and displayed in the plot legend and tracked in Table 3. The initial Oasys MEAN and STDEV are **15.603** and **6.566**, which are significantly distant from Aprisa MEAN and STDEV **25.793** and **16.264**. Finally, we computed the RMSE error for Oasys cell delays, where the average initial Oasys RMSE is **16.958** (Table 3). Second, we employed a 75–25 training–testing split for each design from TC1 to TC4 using the Python Scikit-learn function *train_test_split*. We trained each model using each design separately and extracted the resulting accuracy scores and error rates as presented in Table 5. Our objective was to observe the behavior of each model regarding different design architectures.

Afterward, we created the *Bunch* dataset by combining TC1, TC2, TC3, and TC4 into a single CSV file, which was then shuffled and split into a 75–25 training–validation split. The *Bunch* dataset is referred to as such in Tables 4 and 6. The validation set was used to identify the optimal hyperparameters that result in the highest model accuracy. This was accomplished by employing the *GridSearchCV* function from the scikit-learn library. To further evaluate the model's generalization ability, the models trained on the *Bunch* dataset were tested on new unseen data (TC5) to assess its generalization capability. Hence, TC5 was utilized solely for testing the models trained with the Bunch dataframe (Table 6). In Table 6, we present the R2 score and evaluation metrics corresponding to the validation set using the best estimators provided by the GridSearchCV function and scores obtained after predicting using TC5 data.

5.2. Prediction results

The target delays are predicted using *Linear Regression*, *Decision Tree*, *Random Forest*, *K-Nearest Neighbors*, and *Support Vector Regression*. In Fig. 3, we display the resulting predictions for the testing dataset per model and design. The Figure contains 20 subplots, five models per design. As a result, the predicted values look more consolidated and proximate to the golden value.

Table 5 summarizes each model's accuracy R2 score and the obtained error metrics RMSE and MAE. We note that the models' response differs from one design to another depending on the internal design architecture. The *Random Forest* predictions are the most accurate, with an average accuracy score of 91.25%. The *Random Forest RMSE* and *MAE* as well are the lowest average error with an RMSE and MAE of 4.469 and 1.9, respectively. With a very close accuracy and lower RMSE and MAE, the *KNN* model has the second least average errors of 4.941 and 2.144 and the second-highest accuracy score of 89.28%. The *Decision Tree* also offers a high accuracy score of 87.49% and average errors of 5.212 and 2.529. However, *SVR* and *Linear Regression* are considered inappropriate for our delay prediction, and both show

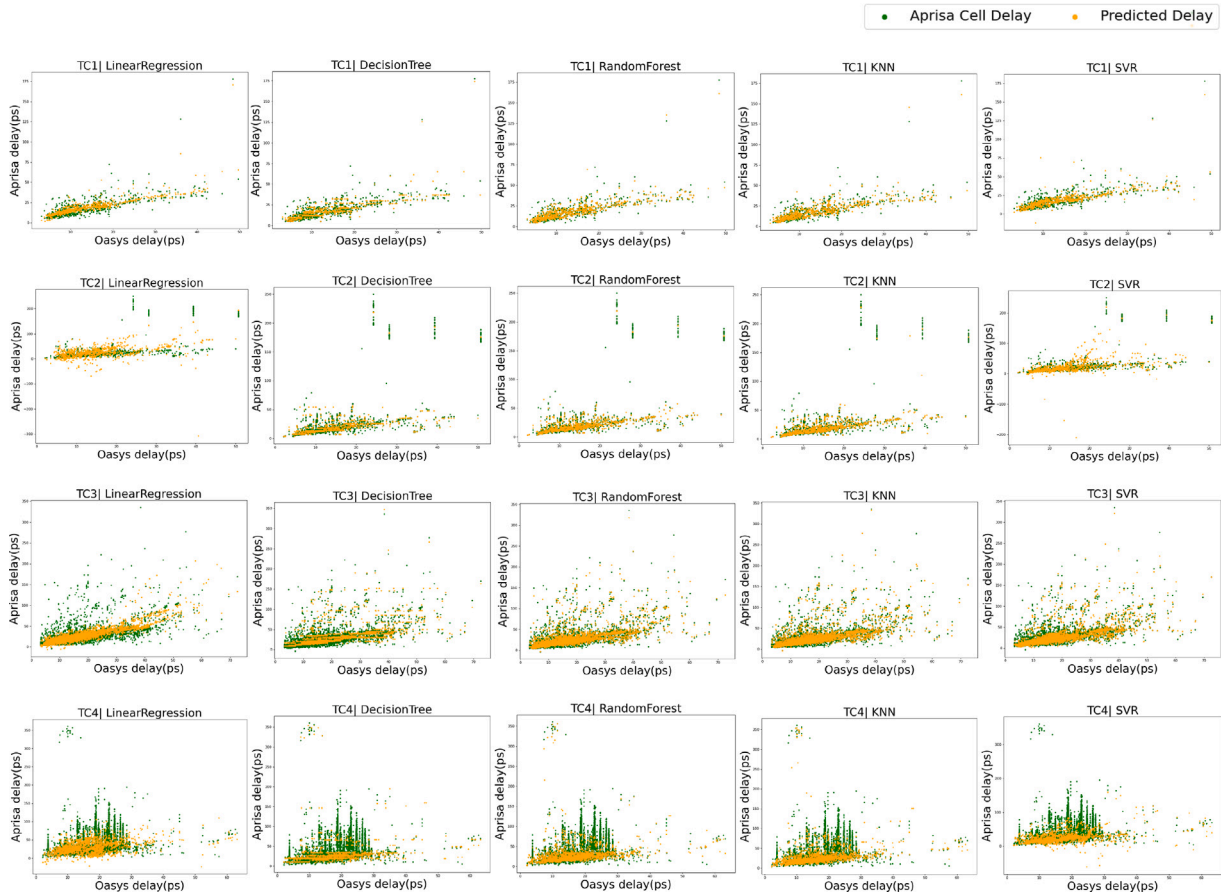

Fig. 3. Scatter plots of predicted cell delay vs. Actual cell delay for each design.

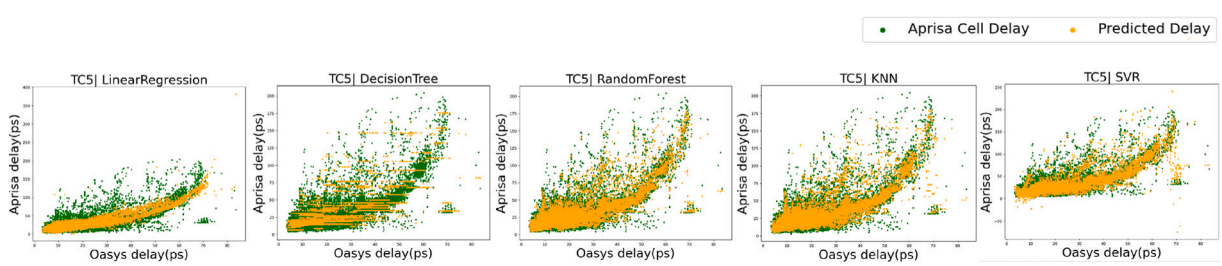

Fig. 4. Scatter plots of predicted cell delay vs. Actual cell delay for TC5 using models trained on bunch dataset.

the lowest accuracy, highest RMSE, and MAE. We visualize scores and RMSE comparison in a histogram as in Figs. 5 and 6.

Similarly, we trained new models using the '*Bunch*' combined dataframe and used TC5 as testing data. The Random Forest model shows the highest accuracy of 89.12% using validation data and 75.72% using testing data. The RF also provides the lowest errors rate of 6.29 and 2.731 for validation data, 6.947 and 3.099 for testing data. Both obtained scores are lower than the average line in Table 5, which is expected because TC5 is entirely new data and may contain outliers that significantly impact the model's predictions. Consequently, the Random Forest model delivered higher stability and predictive capability, reaching a 75.72% accuracy for unseen data is mainly convenient. The accuracy of the model can be enhanced by feeding it with a larger training dataset and by further tuning the random forest with additional hyperparameter configurations. In Fig. 4, we plot the resulting predictions of the testing dataset TC5 per model in 5 subplots. As a result, the predicted values look more consolidated and proximate to the golden value.

Table 7
Model's training and inference time (s).

Design	MLR		DT		RF		KNN		SVR	
	Train.	Inf.	Train.	Inf.	Train.	Inf.	Train.	Inf.	Train.	Inf.
TC1	0.02	0.02	0.02	0.02	0.70	0.06	0.01	0.26	1.74	1.22
TC2	0.28	0.26	0.26	0.2	4.71	0.68	0.23	71.77	293	443
TC3	0.37	0.3	0.35	0.26	12	1	0.26	89.74	861	608
TC4	0.78	0.58	0.69	0.52	14.24	1.71	0.48	360	2182	2331
Bunch	1.23	–	1.17	–	42	–	0.71	–	9639	–
TC5	–	1.13	–	1.31	–	13.33	–	171	–	1840
Reduced Bunch	0.03	0.01	0.15	0.02	14	2.77	30	33	529	789

Even the resulting scores achieved by the *Random Forest*, *KNN*, and *Decision Tree* are among the highest, we compared their training time and inference time. The Random Forest requires a few seconds to train and predict cell delays, depending on the design size. As in Table 7, TC4 lasted around 14 s to train and 1.7 s to predict results. In contrast,

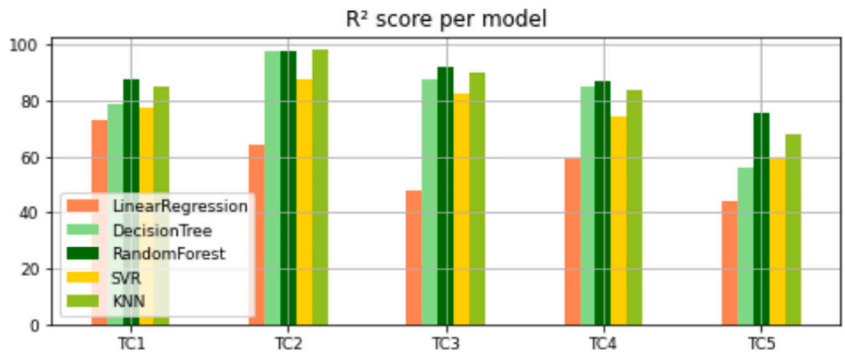

Fig. 5. R2 score per Model.

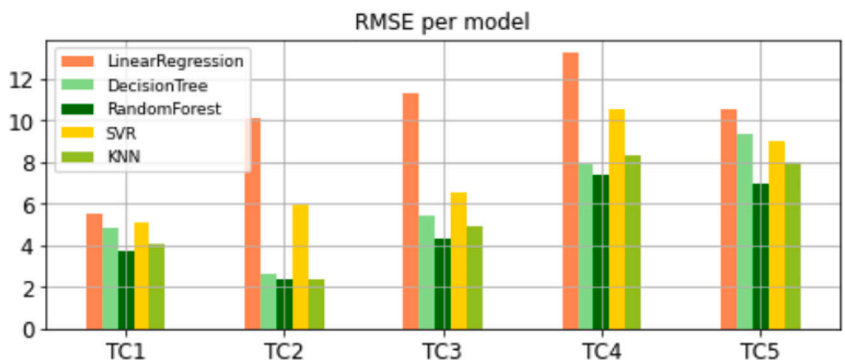

Fig. 6. RMSE per Model.

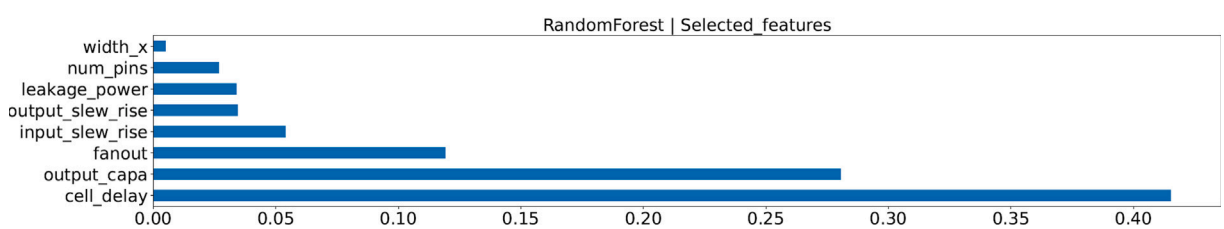

Fig. 7. Random Forest selected features.

Table 8
Initial Oasys versus Predicted metrics.

Design	Initial Oasys			Random-Forest predictions			Aprisa golden	
	MEAN	STDEV	RMSE	MEAN	STDEV	RMSE	MEAN	STDEV
TC1	13.112	7.562	9.083	17.938	10.729	3.761	17.907	11.080
TC2	14.693	4.115	22.525	30.760	17.335	2.343	30.754	17.445
TC3	18.083	8.250	14.785	26.345	15.305	4.362	26.332	15.782
TC4	16.527	6.338	21.442	28.173	19.474	7.413	28.181	20.751
Average	15.603	6.566	**16.958**	**26.105**	**15.184**	**4.469**	25.793	16.264
Bunch	16.495	6.642	**20.069**	**28.129**	**17.902**	**6.290**	28.125	18.877
TC5	20.644	10.916	**12.463**	**27.312**	**13.080**	**6.947**	27.231	14.099

SVR and KNN models last longer and attain a few minutes, exceeding an hour of inference time for the SVR model, which does not align with our end purpose. The Bunch dataframe took a few seconds to predict delays using LR and DT and less than a minute for RF, which perfectly matches our interest in using this model as a framework. As per the used machine, we have run all models using identical CPU and memory.

5.3. Random forest and further analysis

In Table 8, we present the mean, standard deviation, and RMSE of the predictions made by the *Random Forest* model. Notably, the average RMSE for cell delay predictions has decreased to 4.469, which is a significant improvement compared to the original Oasys average RMSE of 16.958. Additionally, the mean and standard deviation of the predicted cell delays are more aligned with those of the Aprisa, with the mean being 26.105 compared to 25.793 and the standard deviation being 15.184 compared to 16.264. Hence, we have achieved a more comparable mean and standard deviation. Similarly, for the *Bunch* dataset, the error has been significantly reduced from 20.069 to 6.290, although it is slightly higher than the average. This pattern is also observed in TC5, where the RMSE error has been reduced from 12.463 to 6.947. At the same time, the mean and standard deviation are closely aligned with those of the Aprisa.

Finally, we performed a feature selection on the testing data to spotlight the highly involved features during delay prediction. The model assign scores to each input feature based on its usefulness during the prediction phase. High-scored features are the most relevant and contribute the most to the prediction. We illustrate in Fig. 7 the selected features as per *Random Forest*. The features ranking indicates the following order: *cell delay* retains the highest weight (42%), then the *output capacitance* (28%), *cell fanout* (12%), *input slew rise* (5%) as well as the *output slew rise* (3%), these are considered the highly contributing features. However, the *cell leakage* and *cell pins number* also have a small contribution.

Table 9
Reduced Bunch Dataframe.

dataset	Columns°				Model purpose	LR		DT		RF		KNN		SVR	
	Int64	Float64	Bool.	Total		R2	RMSE	R2	RMSE	R2	RMSE	R2	RMSE	R2	RMSE
Bunch	19	18	64	101	Training/Testing	47.37	13.838	85.41	7.285	**89.12**	**6.290**	86.67	6.963	85.86	6.285
Reduced Bunch	1	4	–	5		45.00	14.145	81.45	8.214	83.60	7.724	80.61	8.398	**84.89**	**6.361**

5.4. Training with reduced features

In light of the results obtained from the feature selection process, the Random Forest algorithm assigned high scores to the features of *cell delay*, *output capacitance*, *cell fanout*, *input slew rise*, and *output slew rise*. To further assess the impact of these high-scored features on the model's performance, we reduced the *Bunch* dataset to include only these features, as shown in Table 9. The reduced dataset was then shuffled and split into a 75–25 training–testing split. Finally, new models were trained and evaluated. As presented in Table 9, the reduction of features led to a slight decrease in accuracy and increased RMSE. The accuracy of the Random Forest model dropped from 89.12% to 83.60%, which is still considered acceptable. On the other hand, the Support Vector Regression (SVR) model retained its accuracy of 84.89% and emerged as the best estimator model for the reduced dataset. Regarding runtime, the Random Forest algorithm took approximately 15 s to train, while the SVR runtime was significantly reduced from 9639 s to 530 s. We recognize the potential of using the SVR model with reduced datasets. This insight motivates us to tune new hyperparameter configurations for the SVR model, aiming to increase its accuracy further and reduce runtime.

6. Conclusion

In this paper, we studied a Machine Learning application for cell delay prediction for a post-placed gate-level netlist. Our purpose is to improve the accuracy of cell delays at an early design stage. We trained and tested five Machine Learning models with the help of several industrial designs. We have improved Oasys cell delay accuracy based on Aprisa Backend information, which we consider golden data. The proposed ML models achieved a higher prediction accuracy and lower error rate. The *Random Forest* model had the highest accuracy and lowest RMSE. Consequently, we reduced the average RMSE from 16.958 to 4.469 with an average accuracy of 91.25%. The Random Forest also produced higher stability and predictive capability for new design circuits. With more data added to train the Random Forest model, we can achieve an excellent generalization ability. The objective is to use this model integrated into a Python framework right after the logic synthesis stage. The model will provide precise estimations without the need for physical information. Based upon the selected features highlighted in the last chapter, the model will use only attributes within the Oasys synthesis tool. The model will support R&D and QA engineers to detect the tool's irregularity and spotlight outliers engendered during logic synthesis. In terms of runtime, the *Random Forest* model delivers accurate values within a few seconds depending on design complexity, which perfectly matches our purpose. As a future work and extension to this study, we look forward to improving the training dataset with additional large designs and enhancing the overall model's accuracy. We plan to investigate and implement other complex algorithms to enhance the model's performance and integrate it into a comprehensive Python-based framework for wider application and accessibility.

CRediT authorship contribution statement

Yassine Attaoui: Conception and design of study, Acquisition of data, Analysis and/or interpretation of data, Writing – original draft, Writing – review & editing. **Mohamed Chentouf:** Conception and design of study, Acquisition of data, Analysis and/or interpretation of data, Writing – original draft, Writing – review & editing. **Zine El Abidine Alaoui Ismaili:** Conception and design of study, Acquisition of data, Analysis and/or interpretation of data, Writing – original draft, Writing – review & editing. **Aimad El Mourabit:** Conception and design of study, Acquisition of data, Analysis and/or interpretation of data, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgment

All authors approved the version of the manuscript to be published.

References

[1] Siemens EDA Oasys-RTL™ user's guide, in: Oasys-RTL™ User's Guide, Software Version 2021.2.R1, 2021.
[2] Siemens EDA APRISA™ user's guide, in: APRISA™ User's Guide, Release Version 2022, 2022.
[3] M. Chentouf, Z.E.A. Alaoui Ismaili, A novel net weighting algorithm for power and timing-driven placement, VLSI Des. 2018 (2018).
[4] E. Zennaro, L. Servadei, K. Devarajegowda, W. Ecker, A machine learning approach for area prediction of hardware designs from abstract specifications, in: 2018 21st Euromicro Conference on Digital System Design, DSD, IEEE, 2018, pp. 413–420.
[5] J. Zhao, T. Liang, S. Sinha, W. Zhang, Machine learning based routing congestion prediction in FPGA high-level synthesis, in: 2019 Design, Automation & Test in Europe Conference & Exhibition, DATE, IEEE, 2019, pp. 1130–1135.
[6] S. Dey, S. Nandi, G. Trivedi, Machine learning for VLSI CAD: A case study in on-chip power grid design, in: 2021 IEEE Computer Society Annual Symposium on VLSI, ISVLSI, IEEE, 2021, pp. 378–383.
[7] Y. Zhou, H. Ren, Y. Zhang, B. Keller, B. Khailany, Z. Zhang, PRIMAL: Power inference using machine learning, in: Proceedings of the 56th Annual Design Automation Conference 2019, 2019, pp. 1–6.
[8] M. Chentouf, C. Naimy, Z.E.A.A. Ismaili, Machine learning application for early power analysis accuracy improvement: A case study for cells switching power, in: 2021 International Conference on Microelectronics, ICM, IEEE, 2021, pp. 17–20.
[9] A.B. Kahng, U. Mallappa, L. Saul, Using machine learning to predict path-based slack from graph-based timing analysis, in: 2018 IEEE 36th International Conference on Computer Design, ICCD, IEEE, 2018, pp. 603–612.
[10] E.C. Barboza, N. Shukla, Y. Chen, J. Hu, Machine learning-based pre-routing timing prediction with reduced pessimism, in: 2019 56th ACM/IEEE Design Automation Conference, DAC, IEEE, 2019, pp. 1–6.
[11] Z. Guo, M. Liu, J. Gu, S. Zhang, D.Z. Pan, Y. Lin, A timing engine inspired graph neural network model for pre-routing slack prediction, 2022.
[12] R. Liang, Z. Xie, J. Jung, V. Chauha, Y. Chen, J. Hu, H. Xiang, G.-J. Nam, Routing-free crosstalk prediction, in: 2020 IEEE/ACM International Conference on Computer Aided Design, ICCAD, IEEE, 2020, pp. 1–9.
[13] A.B. Kahng, S. Kang, H. Lee, S. Nath, J. Wadhwani, Learning-based approximation of interconnect delay and slew in signoff timing tools, in: 2013 ACM/IEEE International Workshop on System Level Interconnect Prediction, SLIP, IEEE, 2013, pp. 1–8.
[14] S.-S. Han, A.B. Kahng, S. Nath, A.S. Vydyanathan, A deep learning methodology to proliferate golden signoff timing, in: 2014 Design, Automation & Test in Europe Conference & Exhibition, DATE, IEEE, 2014, pp. 1–6.
[15] S. Dai, Y. Zhou, H. Zhang, E. Ustun, E.F. Young, Z. Zhang, Fast and accurate estimation of quality of results in high-level synthesis with machine learning, in: 2018 IEEE 26th Annual International Symposium on Field-Programmable Custom Computing Machines, FCCM, IEEE, 2018, pp. 129–132.

[16] A. Géron, Hands-on Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems, "O'Reilly Media, Inc.", 2019.
[17] G. Hackeling, Mastering Machine Learning with Scikit-Learn, Packt Publishing Ltd, 2017.
[18] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, E. Duchesnay, Scikit-learn: Machine learning in python, J. Mach. Learn. Res. 12 (2011) 2825–2830.
[19] B.F. Darst, K.C. Malecki, C.D. Engelman, Using recursive feature elimination in random forest to account for correlated variables in high dimensional data, BMC Genet. 19 (1) (2018) 1–6.
[20] H.S. Obaid, S.A. Dheyab, S.S. Sabry, The impact of data pre-processing techniques and dimensionality reduction on the accuracy of machine learning, in: 2019 9th Annual Information Technology, Electromechanical Engineering and Microelectronics Conference, IEMECON, IEEE, 2019, pp. 279–283.
[21] Y. Hu, S. Yu, X. Qi, W. Zheng, Q. Wang, H. Yao, An overview of multiple linear regression model and its application, Zhonghua Yu Fang Yi Xue Za Zhi [Chin. J. Prevent. Med.] 53 (6) (2019) 653–656.
[22] M. Rakhra, P. Soniya, D. Tanwar, P. Singh, D. Bordoloi, P. Agarwal, S. Takkar, K. Jairath, N. Verma, Crop price prediction using random forest and decision tree regression:-a review, Mater. Today: Proc. (2021).